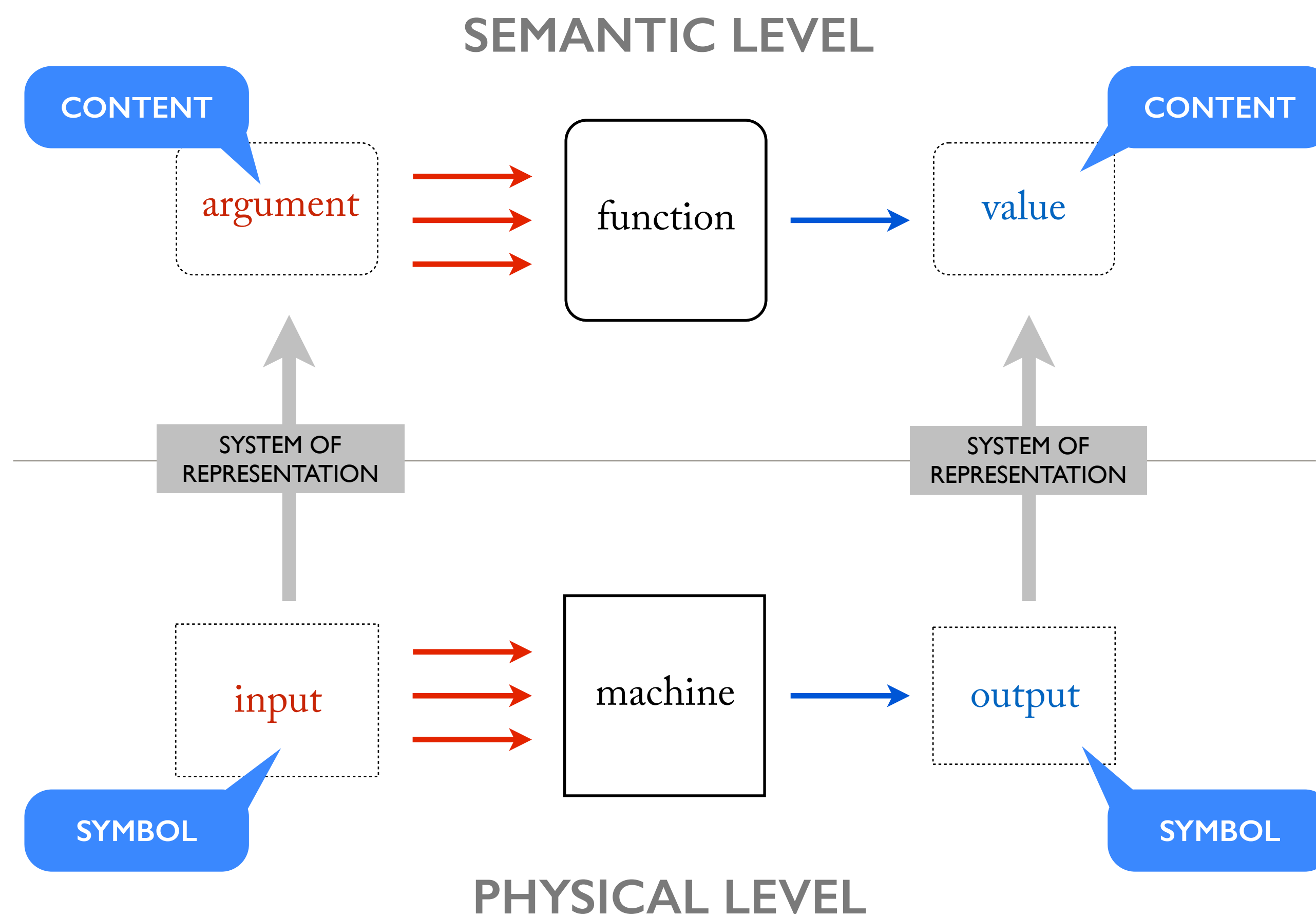


5. Systems of Representation

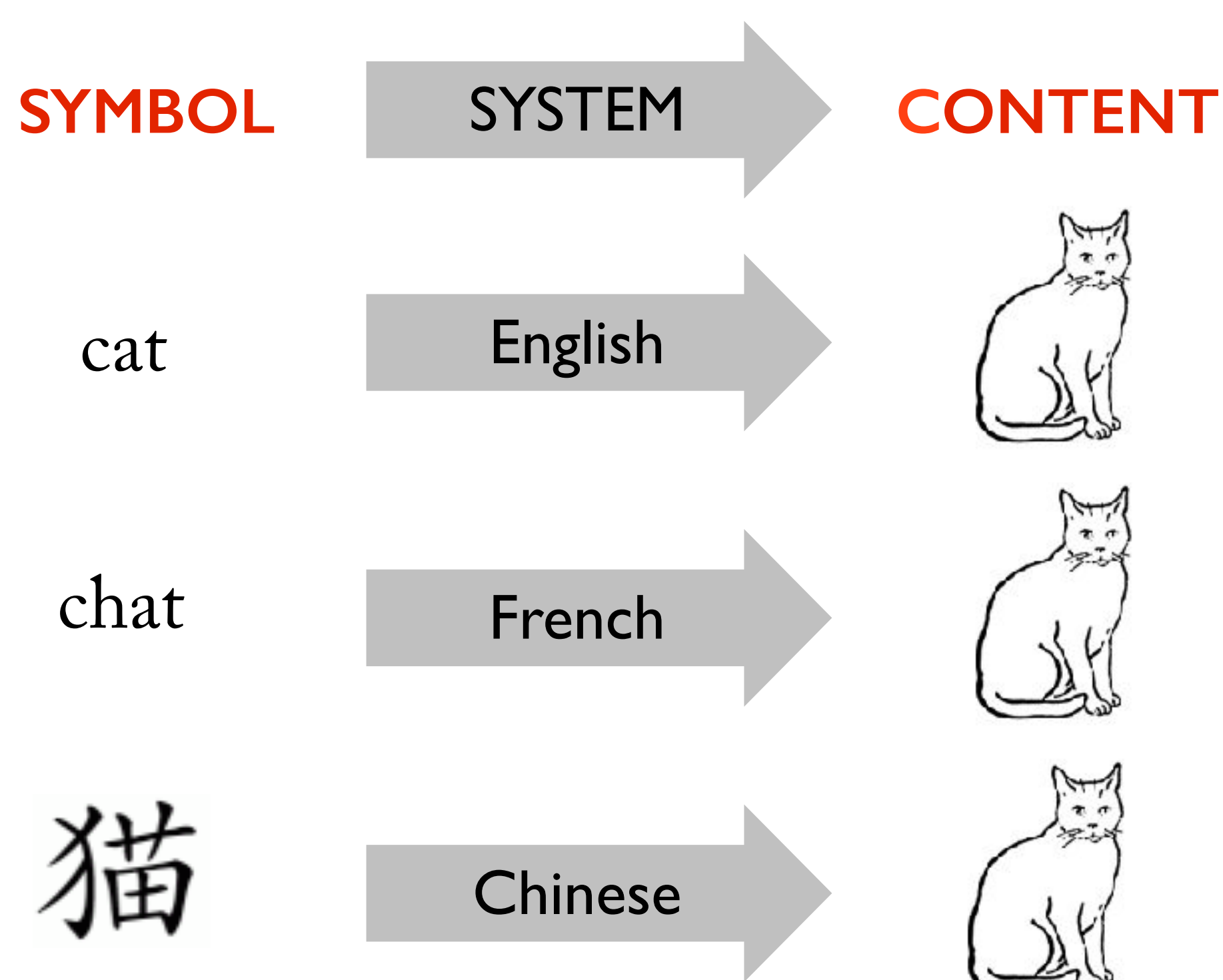
The role of representational systems in computation

Systems of representation associate symbols with contents. In computation, they map inputs to arguments and outputs to values. They are the glue that connects the physical and semantic levels.



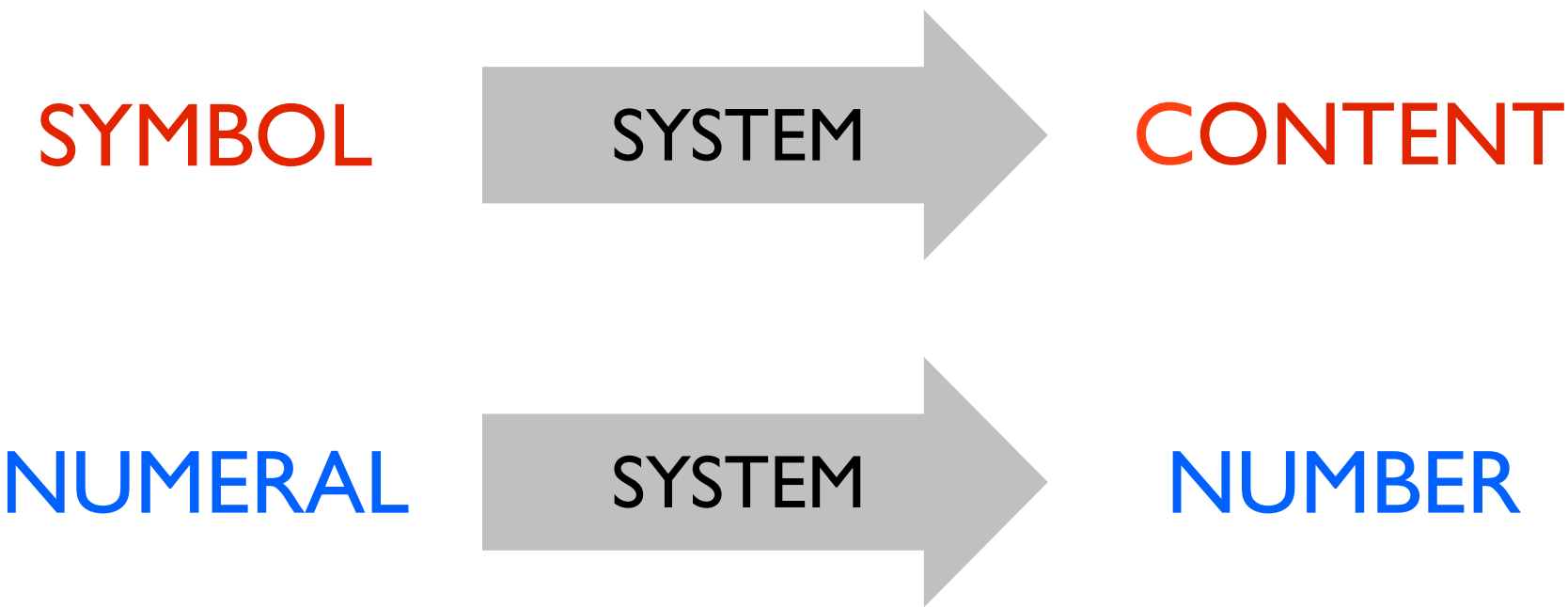
Symbol, content, system

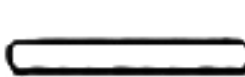
Symbols express contents only relative to a system. The same content can be expressed by different symbols in different systems. Here is an illustration for the case of words in a language.

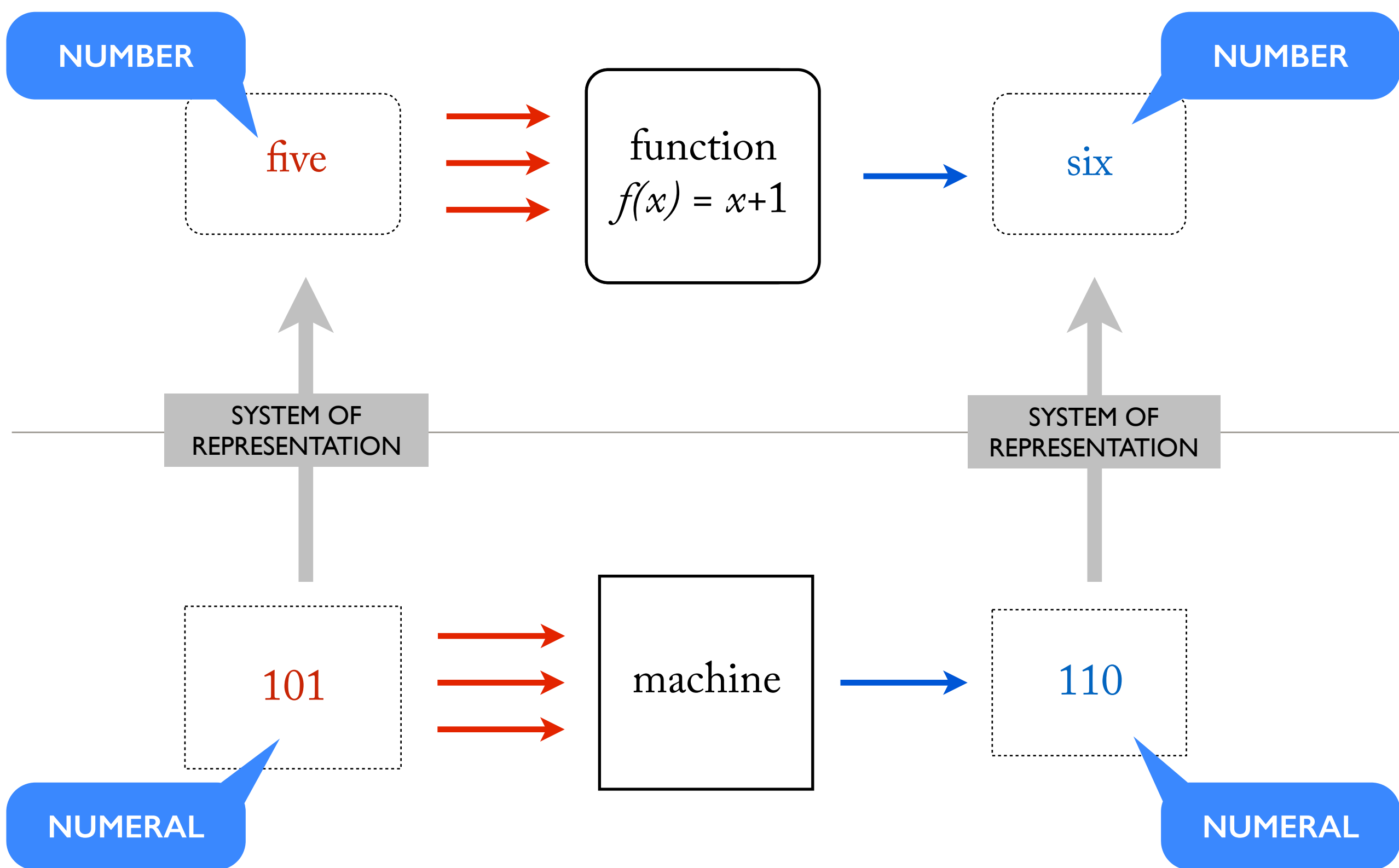


Numeral systems

Numeral systems are systems of representation specifically for numbers. In these systems, the symbols are called **numerals**, and the contents are **numbers**. Numerals are types of physical symbols, while numbers are abstract quantities.

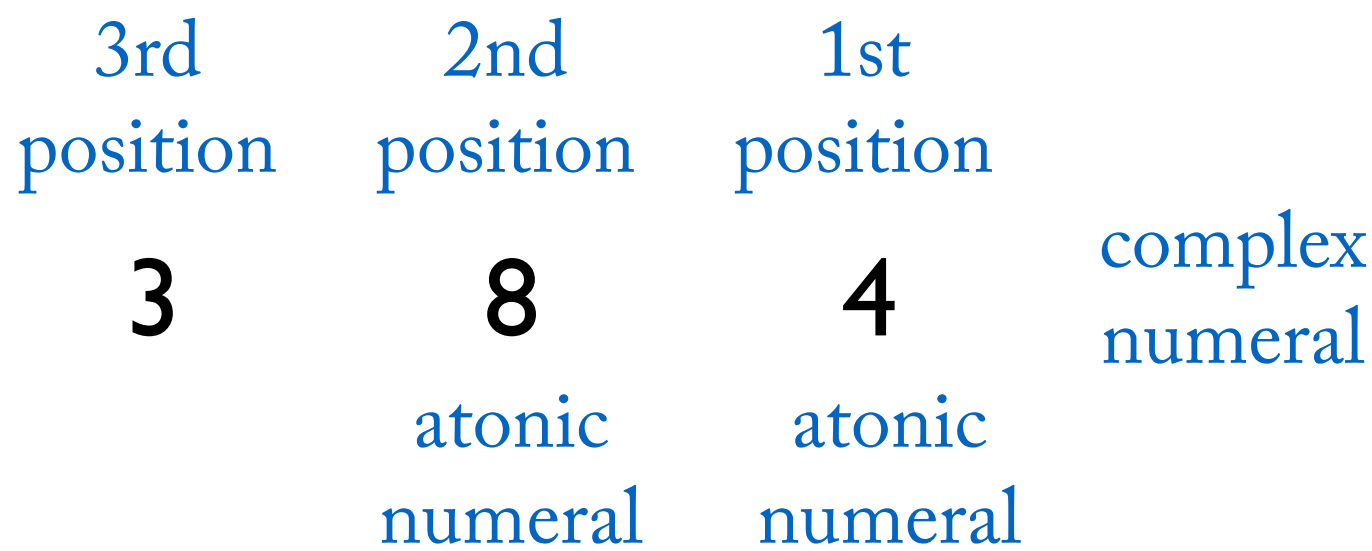


Different numeral systems use....	Roman numerals	Arabic numerals	Mayan numerals	The numbers represented
		0		<i>zero</i>
	I	1		<i>one</i>
	II	2		<i>two</i>
	III	3		<i>three</i>
	IV	4		<i>four</i>
	V	5		<i>five</i>



Anatomy of a Numeral

A **complex** numeral is a made up of a string of **atomic** numerals. Each atomic numeral has a **position** within the complex numeral.



In the the **Indo-Arabic numeral** system we all use, the atomic numerals are the numerals 0-9, and all other numerals are constructed from these.

Atomic symbols in the **Indo-Arabic** system

0 1 2 3 4 5 6 7 8 9

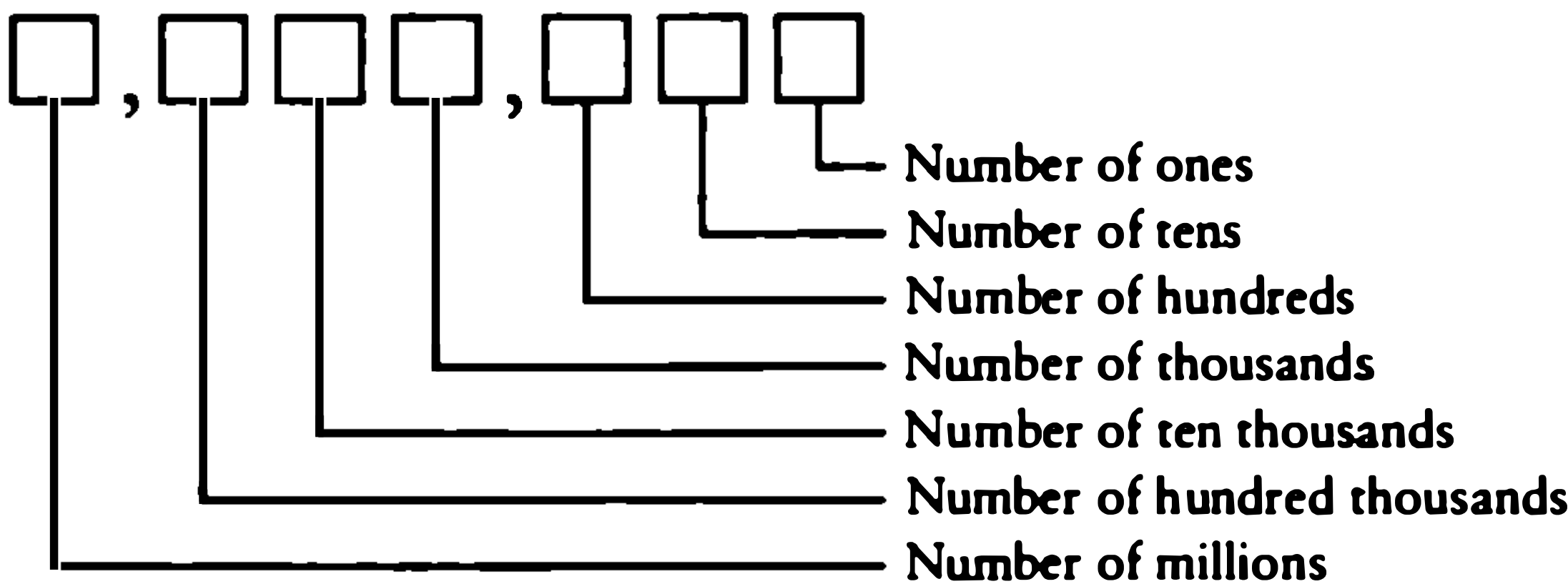
Mechanics of a Numeral System

A **positional** numeral system is defined by

- (i) the **numeric values** it assigns to each **atomic numeral** (starting from zero);
- (ii) the **numerical weight** of the **position rank** (starting from zero),

where the number of symbols in (i) = the weight in (ii).

In the Indo-Arabic system, the **weight** of the position rank is 10. That means that the weight of each position goes up by a power of 10. (The first position is considered position 0, so its weight is $10^0=1$.)



The Arabic numeral system is positional, but the Roman system is not. Both employ complex symbols to express large numbers, but they rely on different forms of representational system.

one hundred and one

101

CI

Arabic numerals

Arabic numerals

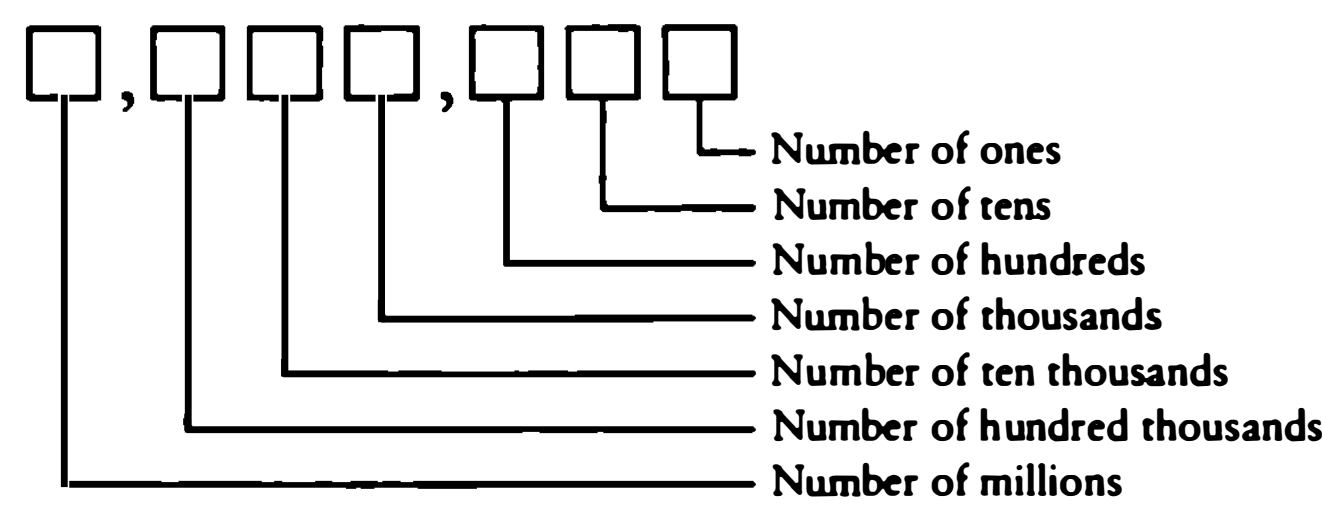
Note that the numeral 1 appears twice in the Indo-Arabic numeral, because it plays different roles in the different positions. In the Roman numeral, which is **additive** rather than positional, the numeral I only appears once, added to the value of C.

Base

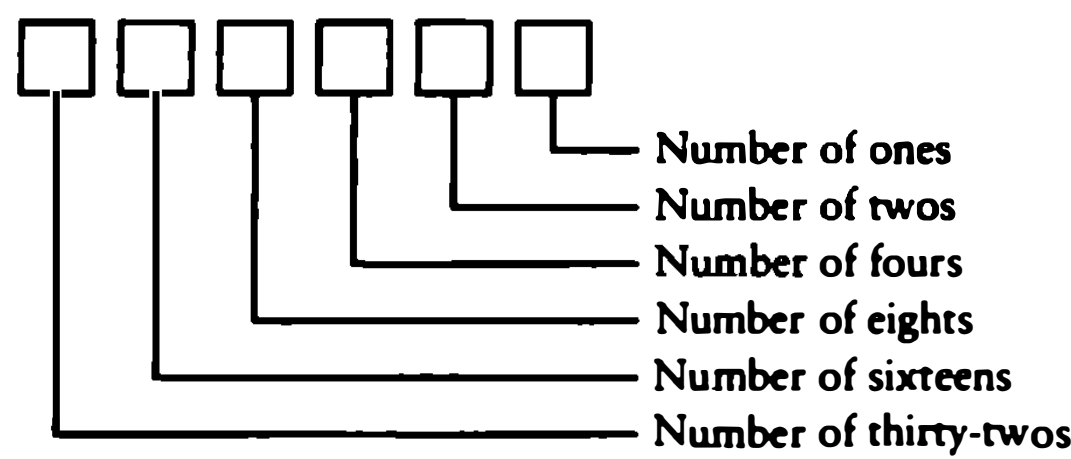
The **base** of a positional system corresponds to the number of atomic symbols. In the Arabic system, there are ten symbols, 0-9, so this system is called **base 10**. In the Binary system, there are only two symbols (0,1), so this system is **base 2**.

The base determines the weight of the positional rank. So in base 10, positions increase in weight by powers of 10, while in base 2, they increase by powers of 2.

Base 10



Base 2 (Binary)



Base 10	Base 7	Base 2 (Binary)	Base 1 (Tally)
1	1	1	
2	2	10	
3	3	11	
4	4	100	
5	5	101	
6	6	110	
7	10	111	
8	...	1000	
9	15	1001	
10	16	1010	
...	20	1011	
98	...	1100	
99	66	1101	
100	100	1110	

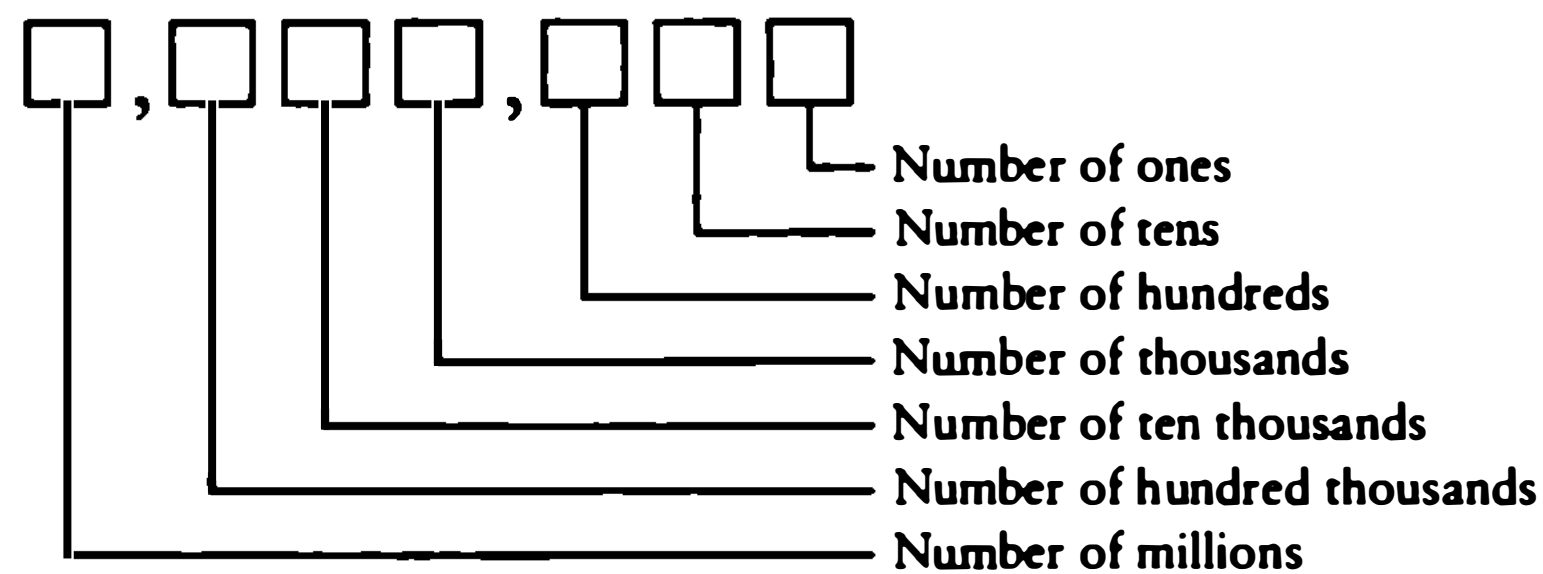
Binary is special because:
two values
size increases non-linearly (~logarithmically)
Today it is the numeral system most used at the mechanical foundations of computing.

Calculating a Numeral's Value

Decimal System

ten atomic symbols, $base=10$

0 1 2 3 4 5 ...
zero one two three four five ...

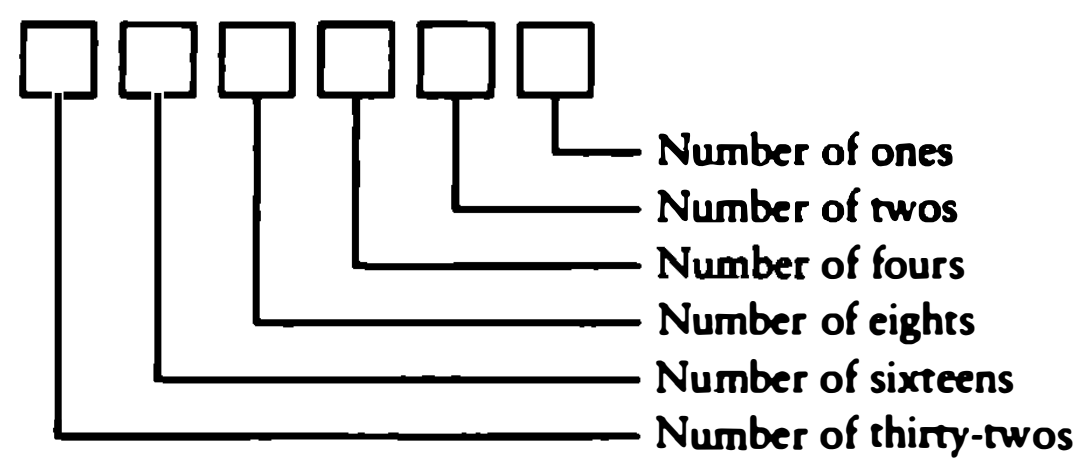


$$111_D = 1_D \times 100 + 1_D \times 10 + 1_D \times 1$$

Binary System

two atomic symbols, $base = 2$

0 1
zero one

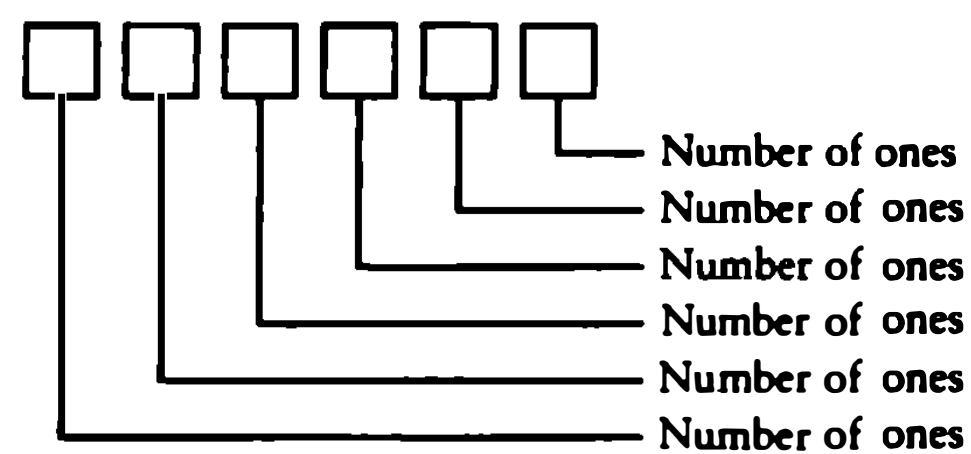


$$111_B = 1_B \times 4 + 1_B \times 2 + 1_B \times 1$$

Tally System

one atomic symbol, $base = 1$

|



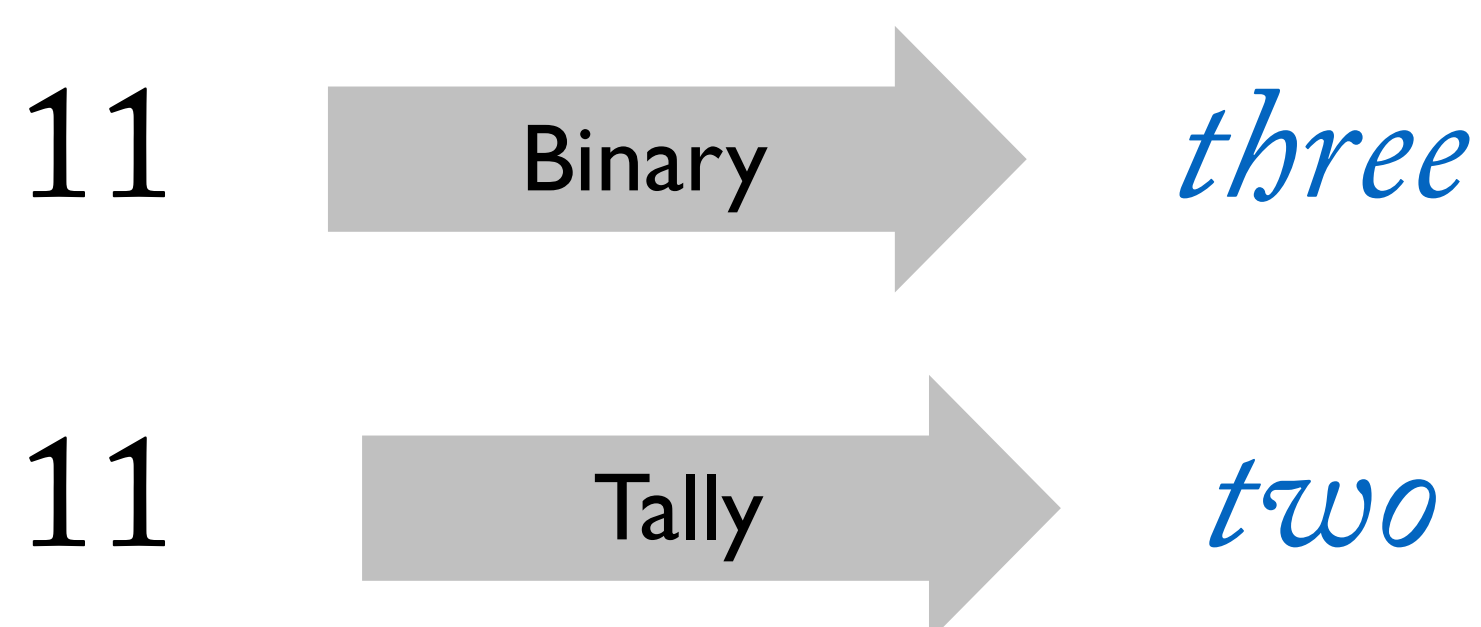
$$111_T = 1_T \times 1 + 1_T \times 1 + 1_T \times 1$$

Note: extended tally system with unlimited 0's to the left.

Binary and Tally

In this course, the most important numeral systems will be **tally (T)** and **binary (B)**. This is because you can express all numbers in these systems using only two symbols, 0 and 1.

The same set of numerals represent different numbers in the two systems.



The **syntax** of a representational system is a set of rules for constructing well-formed complex numerals in that system. When designing machine to compute a function in tally or binary, we'll always assume that inputs are well-formed.

Tally Syntax

Tally systems in common usage take a variety of forms.



Here, a **well formed tally numeral** is any string of bits, where the first n (≥ 0) bits from the right are 1's, and they are followed to the left by any number of 0's.

0 ... 0 1 ... 1

So the following are well formed:

000
111
0011

But these ones are not.

1000
1101
1110

Binary Syntax

A **well formed binary numeral** is any string of bits, where the first n (≥ 0) bits from the right are either 0's or 1's, and they are followed to the left by any number of 0's.

0 ... 0 1|0 ... 1|0

So the following are well formed:

000
111
0011

But these ones are not.

1000
1101
1110